

No Detectable Difference: A Blocked Factorial Experiment

Comparing Claude Fable 5 and GPT-5.6 on Proven-Solvable Coding Tasks

Drake Griffith

Actual Intelligence Labs

`christopherg@actualintelligencelabs.ai`

July 10, 2026

Abstract

We report a controlled comparison of two frontier coding agents: Claude Fable 5 (Anthropic, via the `claude` CLI) and GPT-5.6 “Sol” (OpenAI, via the `codex` CLI). The study comprises 154 runs of a full-factorial designed experiment (model $\times$ reasoning effort $\times$ harness), blocked by task, with replicated runs modeled as Bernoulli trials and seed-interleaved run order. All nine tasks were verified solvable before any model ran. Both models passed every solo run (152/152). A per-cell perfect score of 24/24 carries a 95% Wilson interval of (0.862, 1.000), and the paired head-to-head McNemar test produced zero discordant pairs. We therefore report no detectable difference, a claim distinct from and weaker than equivalence. Raising the reasoning-effort setting changed cost without changing outcomes: the pass rate was 100% at every effort level, while GPT-5.6 at high effort consumed approximately 80% more output tokens than at low effort. An exact sign-flip permutation test on per-task median $\log(\text{total tokens})$ shows Claude Fable 5 substantially cheaper per solved task ($p = 0.0039$), subject to a stated caveat on cross-vendor input-token accounting. Two independent model judges scored all transcripts blind; their scores agreed within one point on 96% of 139 dual-judged runs. The principal finding is a ceiling effect: the suite was too easy to rank the models, and we report that conclusion rather than a ranking the data do not support. We present the full design, exact statistics, power analysis, and limitations, and describe the harder-tier follow-up experiment.

1 Introduction

Most public comparisons of coding models are anecdotal: one operator, one prompt, two tools, and a stated preference. Because large-language-model inference is non-deterministic, a single run measures what happened once rather than what happens on average, and the average is the quantity that matters when an organization chooses which tool to adopt. An informal comparison repeated the next day can reverse its outcome.

Structured public benchmarks face a complementary problem. Frontier models now cluster near the top of the standard suites, which reduces the suites’ power to discriminate between models, and training-data contamination allows a model to recall an answer instead of deriving

one. Jain et al. document both effects for code generation and respond with a time-segmented benchmark of fresh contest problems [1].

We therefore ran a designed experiment rather than a leaderboard: identical tasks and conditions, sufficient replication to separate signal from sampling noise, exact small-sample statistics, and an explicit statement of what the sample size can and cannot detect. The design follows standard design-of-experiments practice in its treatment structure, blocking, and randomization [9].

Two results follow. The first is a ceiling effect: both models passed every solo task, so this suite establishes that both clear the tier but cannot rank them. The second concerns the reasoning-effort setting, which on this tier produced identical outcomes at materially different token cost, a pattern consistent with the literature on inefficient reasoning [2, 3].

2 Experimental design

2.1 Treatment structure

The run matrix is a full-factorial designed experiment:

- **Factors.** Model (Claude Fable 5, GPT-5.6) $\times$ reasoning effort (two to three levels per model) $\times$ harness (bare CLI vs. a shared agent-instruction harness).
- **Blocking.** Task (eight T1/T2 tasks plus one T3 greenfield build). Every configuration sees every task, so task difficulty cancels out of all paired comparisons.
- **Replication.** $n = 3$ repetitions per cell on T1/T2 tasks and $n = 2$ on T3. Because inference is non-deterministic, replicated runs are required to estimate a pass rate; a single run does not constitute a measurement.
- **Randomization.** Run order is seed-interleaved (seed 1337) so that time-of-day and API-load drift cannot systematically favor one model.

The experiment totals 154 official runs: a 120-run sweep 1 (bare configurations, full effort ladder), a sweep 2 adding harnessed conditions and the greenfield build, and a small hybrid arm (Section 5.6).

2.2 Task suite

The suite contains nine tasks, each verified solvable before any model ran:

- **Four seeded-bug tasks** containing subtle, deliberately introduced defects: an off-by-one error in pagination, a naive versus timezone-aware datetime defect, a cache key that omits a dimension, and an asynchronous deduplication race condition.
- **Four feature tickets** written in a strict six-section ticket format.
- **One greenfield CLI build** (`splitcost`, a cost-splitting tool of approximately 200 lines, with a reference implementation).

Each task ships with a self-test: the verifier fails on the broken starting code and passes after the reference patch is applied. All nine self-tests passed before the experiment began, so any observed failure is attributable to the model rather than to an unsolvable task.

2.3 Controlled conditions and blind dual judging

Conditions were byte-identical across arms: the same harness instruction files, the same prompts, and the same deterministic `verify.sh` gate deciding pass or fail. Every transcript and diff was stripped of identifying markers and scored blind by two independent judges, one Claude judge and one GPT judge, on a four-dimension rubric. Dual judging is a design requirement rather than an optional safeguard: LLM judges carry a documented self-preference bias [4], and a single-model judge grading its own model’s work embeds a conflict of interest in the evaluator. Judge agreement is reported as a result in its own right (Section 5.7).

3 Outcome model

Each run is a Bernoulli trial: pass (`verify.sh` exit 0) or fail. A cell’s empirical pass rate $\hat{p} = k/n$ estimates its Bernoulli parameter p ; within-cell scatter reflects the Bernoulli variance $p(1 - p)$ together with model non-determinism. All per-cell sample sizes are small ($n \in \{2, \dots, 26\}$ per cell; $n = 24$ per arm in the primary sweep-1 comparisons), so every test in Section 4 is exact. The one large-sample approximation in this report is the clearly labelled power search of Section 4.4.

4 Statistical methods

All statistics are computed by a dependency-free reference implementation (`runner/stats.py`, Python standard library only) directly from the raw run and judgment logs. The implementation carries unit self-tests against known values (Appendix A).

4.1 Wilson score intervals for per-cell pass rates

For k successes in n trials with $z = z_{0.975} \approx 1.95996$, the 95% Wilson score interval [6] is

$$\frac{\hat{p} + \frac{z^2}{2n}}{1 + \frac{z^2}{n}} \pm \frac{z}{1 + \frac{z^2}{n}} \sqrt{\frac{\hat{p}(1 - \hat{p})}{n} + \frac{z^2}{4n^2}}, \quad \hat{p} = \frac{k}{n}. \quad (1)$$

Unlike the Wald interval $\hat{p} \pm z\sqrt{\hat{p}(1 - \hat{p})/n}$, the Wilson interval retains correct coverage at small n and at $\hat{p} \in \{0, 1\}$, which is the regime of this experiment, where cells post perfect scores. A perfect 24/24 is therefore reported with its interval (0.862, 1.000) rather than as a bare “100%”: the data are consistent with a true failure rate anywhere from zero to approximately 14%.

4.2 McNemar’s exact test for paired pass/fail comparisons

Head-to-head comparisons pair runs by task $\times$ repetition, so the blocking removes task difficulty from the comparison. With b pairs in which arm A passed and arm B failed, and c pairs with the reverse outcome, only the $n_d = b + c$ discordant pairs carry information about a difference. The exact two-sided McNemar test [7] is a binomial test at $\frac{1}{2}$ on the discordant pairs:

$$p = \min\left(1, 2 \sum_{i=0}^{\min(b,c)} \binom{n_d}{i} \left(\frac{1}{2}\right)^{n_d}\right). \quad (2)$$

When $n_d = 0$ the test statistic is undefined, and we report “no evidence of difference” rather than a manufactured p -value.

4.3 Exact sign-flip permutation test on token cost

We compare token cost per solved task with an exact paired permutation test [8]. For each task t we take the median of $\log(\text{total tokens})$ over passing runs for each model, form the paired difference $d_t = \tilde{m}_t^{\text{Fable}} - \tilde{m}_t^{\text{Sol}}$, and enumerate all 2^k sign assignments over the k tasks:

$$p = \frac{\#\{s \in \{-1, +1\}^k : |\sum_t s_t d_t| \geq |\sum_t d_t|\}}{2^k}. \quad (3)$$

The log transform converts the statistic to a ratio comparison and reduces the influence of the heavy right tail of token counts; medians over passing runs keep the comparison on a per-solved-task basis.

4.4 Power: the minimum detectable effect

The minimum detectable effect (MDE) is found by a search over the normal-approximation power of a two-sided two-proportion z -test, the one deliberate approximation in this report. For per-arm size n , baseline p_1 , alternative $p_2 = p_1 + \delta$, and $\bar{p} = (p_1 + p_2)/2$,

$$\text{power}(\delta) = \Phi\left(\frac{\sqrt{n}|\delta| - z_{\alpha/2}\sqrt{2\bar{p}(1-\bar{p})}}{\sqrt{p_1(1-p_1) + p_2(1-p_2)}}\right), \quad (4)$$

and the MDE is the smallest δ with $\text{power}(\delta) \geq 0.80$ at $\alpha = 0.05$. At $n = 24$ per arm and $p_1 = 0.5$ this gives $\text{MDE} \approx 36.7$ percentage points. The reporting rule follows: a gap smaller than the MDE receives a confidence interval, not a verdict. We report “no detectable difference” and do not report equivalence.

5 Results

5.1 Pass rates

Sweep 1 produced 120 passes in 120 attempts across both models and every effort level; including all solo sweep-2 runs gives **152/152** passes for every configuration in which a model worked alone. Table 1 reports every cell with its Wilson interval (Eq. 1); Figure 1 plots the same intervals.

model	effort	harness	pass/ n	pass rate	95% Wilson CI
Fable	medium	bare	26/26	100%	(0.871, 1.000)
Fable	medium	harness	14/14	100%	(0.785, 1.000)
Fable	high	bare	24/24	100%	(0.862, 1.000)
Hybrid	medium	harness	2/2	100%	(0.342, 1.000)
Sol	low	bare	26/26	100%	(0.871, 1.000)
Sol	low	harness	14/14	100%	(0.785, 1.000)
Sol	medium	bare	24/24	100%	(0.862, 1.000)
Sol	high	bare	24/24	100%	(0.862, 1.000)

Table 1: Per-cell pass rates with 95% Wilson score intervals. Every cell is perfect; the interval width quantifies the small- n uncertainty behind each “100%.” For the hybrid cell, see the disclosure in Section 5.6.

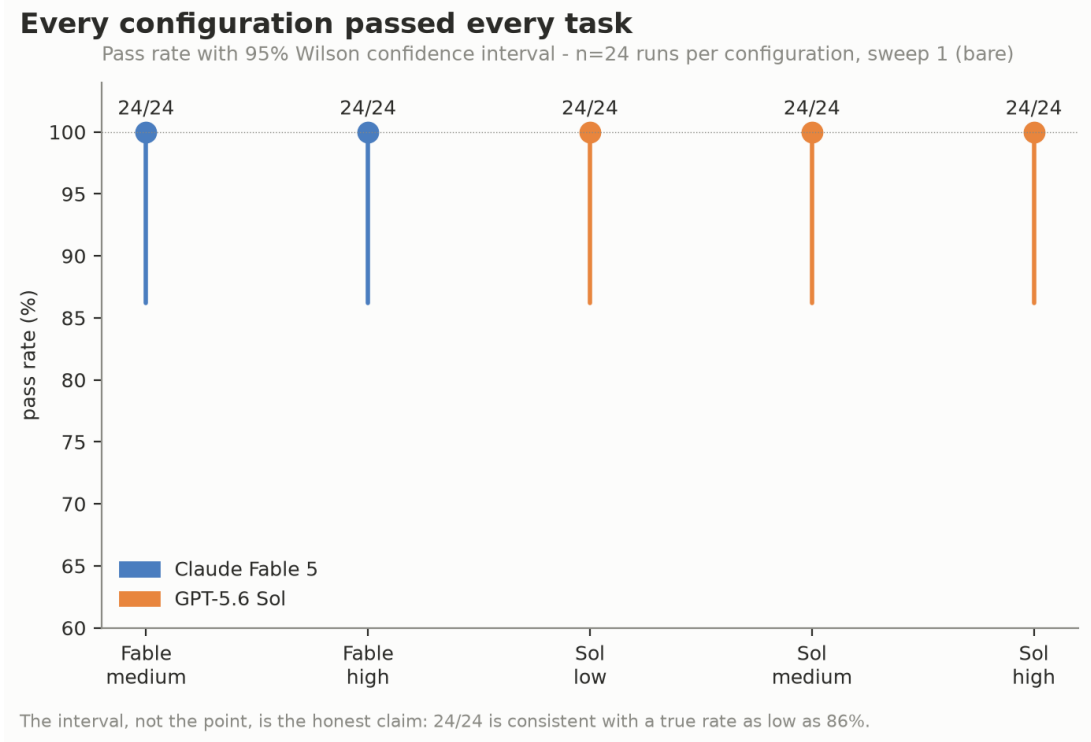


Figure 1: Pass rate by configuration with 95% Wilson intervals. Every bar sits at 100%, with a lower bound determined by the per-cell sample size.

Head-to-head. Each model’s best effort was selected by pass rate with a fewer-tokens tiebreak (Fable: medium; Sol: low), and the 26 task×rep pairs were compared with McNemar’s exact test (Eq. 2). All 26 pairs were pass/pass: $b = c = 0$, so the test statistic is undefined and the comparison carries no evidence of a difference. We conclude **no detectable difference** between the models on this suite, a claim deliberately weaker than equivalence.

5.2 Effect of the reasoning-effort setting

The pass rate was 100% at every effort setting for both models, and every adjacent-rung McNemar comparison (Fable high vs. medium; Sol medium vs. low and high vs. medium; 24 pairs each) produced zero discordant pairs. Cost moved instead (Table 2, Figure 2): Claude Fable 5 at high effort used 16% more output tokens than at medium (2,008 vs. 1,736 median), and GPT-5.6 at high effort used 80% more output tokens than at low (1,966 vs. 1,095) together with approximately $1.8\times$ the input tokens (174.7k vs. 95.2k). On tasks of this tier, higher effort settings added cost without changing any outcome. This pattern is consistent with the published work on inefficient reasoning: reasoning models gauge problem difficulty poorly and produce elaborated output on easy problems without improving correctness [2, 3].

config	pass	output tok	input tok	wall clock	turns
Fable / medium	24/24	1,736	19,412	68.2 s	7
Fable / high	24/24	2,008	19,398	58.8 s	6
Sol / low	24/24	1,095	95,248	31.9 s	1
Sol / medium	24/24	1,534	138,252	49.0 s	1
Sol / high	24/24	1,966	174,662	63.7 s	1

Table 2: Sweep-1 configurations (medians, $n = 24$ per row). Input tokens are CLI-reported and are not comparable across vendors (Section 5.5).

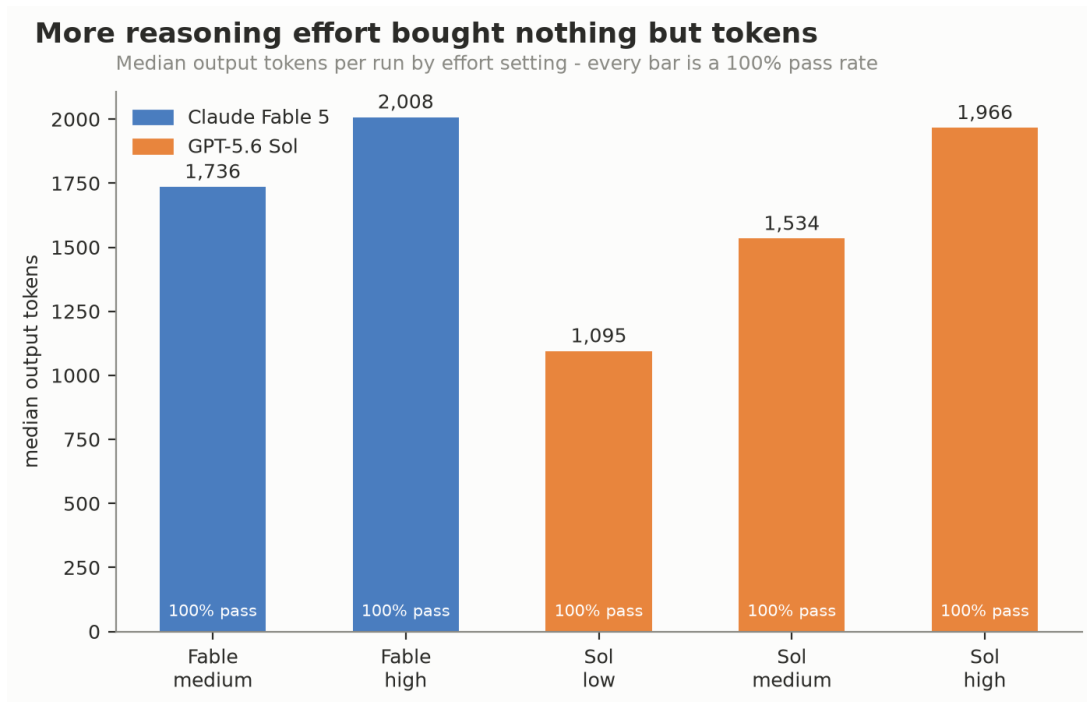


Figure 2: Output tokens by effort level. Token counts rise with the effort setting while every marker remains a 100% pass.

5.3 Solution styles

The two models solve problems in distinct ways. Claude Fable 5 iterates: median 6 to 7 turns per task at approximately 59 to 68 seconds of wall-clock time, alternating between reading, editing, and checking. GPT-5.6 solves in a single pass: median 1 turn at approximately 26 to 64 seconds, reasoning internally and emitting one solution. Neither style produced better outcomes, because both cleared the tier. Turn count is a workflow signature, not a quality signal.

5.4 Effect of the harness

Paired bare-vs-harnessed comparisons at each model’s winning effort (14 pairs each) again produced zero discordant pairs: 100% $\rightarrow$ 100%. The harness was designed for longer and less structured work than these tasks. On this suite it changed token spend (Fable: -560 mean tokens; Sol: $+71,624$, driven by its instruction-context handling) but no outcomes.

5.5 Token cost per solved task

Table 3 reports the exact sign-flip permutation test (Eq. 3) on per-task median $\log(\text{total tokens})$ over passing runs. All nine paired differences favor Claude Fable 5; the observed sum of differences is -17.72 , and 2 of $2^9 = 512$ sign patterns are as extreme or more extreme, giving two-sided $p = 0.0039$. By its own CLI’s accounting, Claude Fable 5 used roughly 6 to $10\times$ fewer total tokens per solved task on this suite.

task	Fable med $\log(\text{tok})$	Sol med $\log(\text{tok})$	diff (F–S)
t1-py-a	9.9293	11.7945	-1.8652
t1-py-b	9.9256	11.7996	-1.8739
t1-ts-a	9.9218	11.7104	-1.7886
t1-ts-b	9.9579	11.9564	-1.9985
t2-py-a	9.9333	11.6615	-1.7282
t2-py-b	9.9590	12.0185	-2.0595
t2-ts-a	10.0253	12.2987	-2.2734
t2-ts-b	9.9256	11.9755	-2.0499
t3-a	9.8712	11.9511	-2.0799
sum of diffs = -17.7172 ; 2 of 512 patterns as or more extreme $p = 0.0039$			

Table 3: Exact sign-flip permutation test on per-task median $\log(\text{total tokens})$ over passing runs, paired by task.

Caveat: input tokens do not compare across vendors. Claude Fable 5’s CLI reports approximately 19.4k median input tokens; GPT-5.6’s reports 95k to 175k. This does not indicate that GPT-5.6 reads five to nine times more code: the two CLIs count context resends and cache reads differently, so their input-token figures measure different quantities under the same label (Figure 3). Within-model comparisons are sound because each CLI is self-consistent; cross-model input-token or dollar comparisons are indicative only, and we do not publish them as firm claims. The cross-comparable axes are output tokens, turns, and wall-clock time, and those are the axes this report relies on.

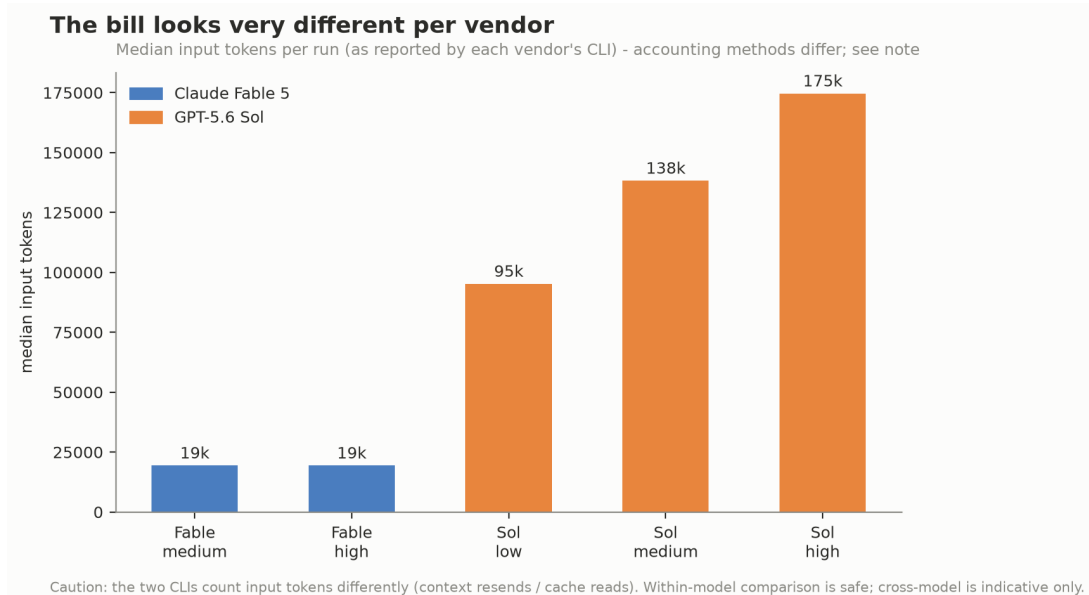


Figure 3: Input tokens by configuration. The cross-model gap is an artifact of CLI accounting rather than workload; the annotation on the chart states the caveat.

5.6 The hybrid configuration

We also tested a hybrid configuration, Claude Fable 5 orchestrating GPT-5.6 as the implementer, on the greenfield task. It was the only configuration that did not run clean. Across four attempts, two produced planning notes and no code, and two shipped working solutions of approximately 160 to 176 lines: **2 of 4**. The official results file records the two later, successful attempts; we disclose the two earlier failures here. The rig was identical across all four attempts, so the most defensible reading is run-to-run variance in a multi-agent handoff rather than a fixed defect. At $n = 4$ we draw no stronger conclusion; the observation motivates a dedicated arm in the next experiment.

5.7 Judge agreement

Across 139 dual-judged runs, with each judge’s score averaged over its four rubric axes, the judges show a mean absolute gap of **0.53** points on a 0–10 scale, Pearson $r = 0.76$, and **96%** of runs within ± 1 point. On the pilot, the GPT judge scored approximately 0.5 points higher than the Claude judge on the same diffs, and both judges scored GPT-5.6’s code slightly higher. These observations are the self-preference risk that motivated blind, dual judging [4], and they are the reason we report the disagreement rather than averaging it away.

6 Limitations

1. **Small per-arm samples.** At $n = 24$ per arm the design has 80% power only for gaps of roughly 36.7 percentage points or more at a 50% baseline (Eq. 4). A smaller real difference receives a confidence interval, not a verdict.

2. **Ceiling effect.** Because both models passed everything, the suite cannot rank them; it establishes only that both clear this tier. Reading “154 perfect runs” as “the models are identical” over-reads the data.
3. **Model judges, not humans.** The judging is blind and dual, with disagreement reported, but the graders remain automated systems with their own biases.
4. **One re-run.** A single Claude Fable 5 greenfield run hit a transient CLI error (2.3s, zero tokens) and was re-run. We disclose it here.
5. **AI-assisted task authoring.** The tasks were verified solvable, but their difficulty was mis-calibrated low. That miscalibration is the principal finding of the study, not a footnote to it.
6. **No dollar claims.** Cost figures use list-price placeholders over vendor-incomparable token counts, and we do not publish them as dollar claims.
7. **Long-context asymmetry.** Practitioner reports and published long-context research find that model quality degrades as context grows large, on the order of 100k tokens [5]. By its own CLI’s accounting, GPT-5.6 exceeded 100k cumulative input tokens on 65 of its 88 runs (median 142k, max 353k); Claude Fable 5 peaked at 19.6k. Two qualifiers apply: the **codex** CLI counts context resends across internal steps, so live context occupancy at any moment is not observable from our data; and the bias runs against GPT-5.6, which passed every task regardless. The tie is therefore unaffected, but blind-judge quality comparisons carry this asymmetry, and we place it on the record.

7 Future work

A ceiling result identifies the direction of the next experiment. Experiment 2 is a hard tier (T4): the same rig, blocking, blind dual judging, and exact statistics, with task difficulty raised until failures appear. Failures are the requirement: a gap becomes measurable only once at least one model begins to miss, and the effort setting can justify its cost only once a higher setting changes an outcome rather than a token count.

Acknowledgments

The experiment harness, task suite, dual-judge pipeline, and statistical appendix are maintained in a private repository. The statistical methodology is grounded in the author’s industrial-engineering coursework (Georgia Tech ISyE): hypothesis testing, confidence intervals, design of experiments, and stochastic processes.

A Reproducibility

Every number in this report is produced by a dependency-free script from the raw logs:

```
python3 runner/stats.py --results runner/results/results.jsonl
--judgments runner/results/judgments.jsonl
```

The script emits the statistical appendix (per-cell Wilson CIs, every McNemar 2×2 table with its exact p -value, the token permutation test, judge agreement, and the power statement) and carries a self-test mode (`--selftest`) checking the primitives against known values: McNemar exact $p(b=1, c=5) = 0.21875$; Wilson 95% CI for $8/10 = (0.4901, 0.9433)$; sign-flip p for eight positive differences $= 2/256 = 0.0078125$; Pearson $r = 1.0$ on exact linear data. All checks pass.

Judging is blind and dual: transcripts and diffs are stripped of identity before either judge scores them, and both a Claude judge and a GPT judge score every run on the same four-dimension rubric.

References

- [1] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv:2403.07974*, 2024.
- [2] Y. Sui et al. Stop overthinking: A survey on efficient reasoning for large language models. *arXiv:2503.16419*, 2025.
- [3] A. Su et al. Between underthinking and overthinking: An empirical study of reasoning length and correctness in LLMs. *arXiv:2505.00127*, 2025.
- [4] K. Wataoka, T. Takahashi, and R. Ri. Self-preference bias in LLM-as-a-judge. *arXiv:2410.21819*, 2024.
- [5] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. *arXiv:2307.03172*.
- [6] E. B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.
- [7] Q. McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.
- [8] R. A. Fisher. *The Design of Experiments*. Oliver and Boyd, Edinburgh, 1935.
- [9] D. C. Montgomery. *Design and Analysis of Experiments*. Wiley, 9th edition, 2017.